

# UNIVERSE.ASM: Codec Assembly Language

## Cross-Reference with Matrix Algebra Opcodes

Jaroslav Petrina

December 2025

### Abstract

This document presents the complete assembly language implementation of the six-register computational substrate (UNIVERSE.ASM) and establishes precise correspondence with the nine codec matrix operators ( $M_1$  through  $M_9$ ). The assembly reveals that codecs 7–10 are architectural composites rather than fundamental operations, while codecs 1–6 map directly to the six fundamental forces emerging from the register algebra.

## Contents

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>1</b> | <b>Architecture Overview</b>                                 | <b>2</b> |
| 1.1      | Hardware Registers (Memory-Mapped I/O)                       | 2        |
| 1.2      | Codec Registers (Virtual)                                    | 2        |
| 1.3      | Working Registers                                            | 2        |
| <b>2</b> | <b>Main Universe Loop</b>                                    | <b>3</b> |
| <b>3</b> | <b>Codec Handlers and Matrix Correspondence</b>              | <b>3</b> |
| 3.1      | Codec 1: Scalar Field (Higgs) — $M_1(\lambda_s)$             | 3        |
| 3.2      | Codec 2: Vector Emission (Photon) — $M_2(f)$                 | 4        |
| 3.3      | Codec 3: Tensor Delay (Graviton) — $M_3(g, \kappa)$          | 4        |
| 3.4      | Codec 4: Decoherence — $M_9(\Delta S)$                       | 4        |
| 3.5      | Codec 5: Strong Confinement — $M_5(\Lambda_s, \chi)$         | 5        |
| 3.6      | Codec 6: Identity Rewrite (Weak) — $M_4(\theta_w, \Gamma_w)$ | 5        |
| <b>4</b> | <b>Composite Codecs (7–10)</b>                               | <b>6</b> |
| 4.1      | Codec 7: Collapse — $M_6(S) + M_9$                           | 6        |
| 4.2      | Codec 8: Black Hole — $M_3$ saturation + time dilation       | 6        |
| 4.3      | Codec 9: Entanglement — $M_7$ phase correlation              | 6        |
| 4.4      | Codec 10: Cosmic Expansion                                   | 7        |
| <b>5</b> | <b>Special Mechanisms</b>                                    | <b>7</b> |
| 5.1      | Pauli Exclusion (Codec 6 Trigger)                            | 7        |
| 5.2      | Fermion 720° Phase Cycle                                     | 7        |
| <b>6</b> | <b>Consistency Cross-Check</b>                               | <b>8</b> |
| 6.1      | Matrix-to-Assembly Mapping                                   | 8        |
| 6.2      | Architectural Constants                                      | 8        |
| 6.3      | Execution Sequence Consistency                               | 8        |
| 6.4      | Critical Distinction: Registers vs Codecs                    | 9        |
| <b>7</b> | <b>Performance Characteristics</b>                           | <b>9</b> |
| 7.1      | Cost Hierarchy                                               | 9        |
| 7.2      | Throughput Limit                                             | 9        |
| <b>8</b> | <b>Conclusion</b>                                            | <b>9</b> |

# 1 Architecture Overview

## 1.1 Hardware Registers (Memory-Mapped I/O)

| Address | Register         | Function                                             |
|---------|------------------|------------------------------------------------------|
| 0x0000  | TIME_LAYER_BUS   | Global Planck tick clock at $1.86 \times 10^{43}$ Hz |
| 0x0001  | IDENTITY_GATE    | Main processor state (Id register)                   |
| 0x0002  | LOAD_ACCUMULATOR | Bus load counter (L register, gravity)               |

Table 1: Core hardware registers

## 1.2 Codec Registers (Virtual)

**Note:** Codec registers are *virtual*—they are combinations of the six working registers. Codecs 7–10 are architectural composites demonstrating machine behavior.

| Address | Codec            | Matrix                    | Function                      |
|---------|------------------|---------------------------|-------------------------------|
| 0x0010  | CODEC1_SCALAR    | $M_1(\lambda_s)$          | Scalar mass field (Higgs)     |
| 0x0011  | CODEC2_VECTOR    | $M_2(f)$                  | EM field (photons)            |
| 0x0012  | CODEC3_TENSOR    | $M_3(g, \kappa)$          | Gravity (tensor delays)       |
| 0x0013  | CODEC4_DECOHERE  | $M_9(\Delta S)$           | Inflation/decoherence         |
| 0x0014  | CODEC5_CONFIN    | $M_5(\Lambda_s, \chi)$    | Strong force (gluon trap)     |
| 0x0015  | CODEC6_REWRITE   | $M_4(\theta_w, \Gamma_w)$ | Weak force (identity rewrite) |
| 0x0016  | CODEC7_COLLAPSE  | <i>composite</i>          | Instantaneous decoherence     |
| 0x0017  | CODEC8_BLACKHOLE | <i>composite</i>          | Gate saturation               |
| 0x0018  | CODEC9_ENTANGLE  | <i>composite</i>          | Entanglement sync             |
| 0x0019  | CODEC10_COSMIC   | <i>composite</i>          | Cosmic expansion              |

Table 2: Codec register map with matrix correspondence

## 1.3 Working Registers

These are the **six fundamental registers** from which all codecs are composed:

| Address  | Register         | Matrix Element    |
|----------|------------------|-------------------|
| 0x0020   | REG_ENTROPY      | $S$ (Entropy)     |
| 0x0021   | REG_COHERENCE    | $C$ (Coherence)   |
| 0x0022   | REG_IDENTITY     | Id (Identity)     |
| 0x0023   | REG_PHASE        | $\Phi$ (Phase)    |
| 0x0024   | REG_SPATIAL      | $X$ (Spatial)     |
| (0x0002) | LOAD_ACCUMULATOR | $L$ (Load/Energy) |

Table 3: Six working registers mapping to matrix vector  $\mathbf{R}$

The complete register vector:

$$\mathbf{R} = \begin{bmatrix} \text{Id} \\ \Phi \\ X \\ S \\ C \\ L \end{bmatrix} \in \mathbb{R}^6 \quad (1)$$

## 2 Main Universe Loop

The fundamental cycle running at Planck frequency:

**Main execution loop:**

```

MAIN_LOOP:
; 1. TICK: Wait for next Planck pulse
WAIT_TICK:
    IN AL, TIME_LAYER_BUS      ; Read time layer
    TEST AL, #0x01             ; Check for tick
    JZ WAIT_TICK               ; Wait until tick arrives

; 2. POLL: Check all codecs for events
POLL_CODECS:
    MOV BX, #0x0010            ; Start at first codec
    MOV CX, #10                ; 10 codecs to check
CODEC_LOOP:
    IN AL, [BX]                ; Read codec state
    TEST AL, #0x80             ; Check event flag
    JZ NEXT_CODEC
    CALL PROCESS_EVENT         ; Apply matrix transform
NEXT_CODEC:
    INC BX
    LOOP CODEC_LOOP

; 3. LOAD: Update bus load (gravity)
UPDATE_LOAD:
    IN AL, LOAD_ACCUMULATOR
    INC AL                     ; L += 1 per tick
    OUT LOAD_ACCUMULATOR, AL

; 4. ENTROPY: Check critical level
ENTROPY_CHECK:
    IN AL, REG_ENTROPY
    CMP AL, CRIT_ENTROPY
    JC MAIN_LOOP
    CALL COSMIC_BLINK          ; Reset on overflow

JMP MAIN_LOOP                  ; Infinite loop

```

## 3 Codec Handlers and Matrix Correspondence

### 3.1 Codec 1: Scalar Field (Higgs) — $M_1(\lambda_s)$

$$M_1(\lambda_s) = \begin{bmatrix} 1 & 0 & 0 & -\ell_1 & +\sigma_1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & (1 - \lambda_s) & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 + \mu_1) & 0 \\ 0 & 0 & 0 & \eta_1 & 0 & 1 \end{bmatrix} \quad (2)$$

**Scalar field handler:**

```

HANDLE_SCALAR:
; Check coherence threshold
IN AL, REG_COHERENCE
CMP AL, #0x80                ; Threshold
JC SCALAR_DECAY

; Emit Higgs boson
MOV AL, #0x7D                ; 125 GeV
OUT CODEC1_SCALAR, AL

; Update: C -> C + mu_1, S -> (1-lambda_s)*S
IN AL, REG_COHERENCE
SUB AL, #0x10                ; C boost
OUT REG_COHERENCE, AL
JMP EVENT_DONE

SCALAR_DECAY:
IN AL, REG_COHERENCE
DEC AL                        ; Gradual decay
OUT REG_COHERENCE, AL

```

**Matrix operation:**  $S \rightarrow (1 - \lambda_s)S$ ,  $C \rightarrow (1 + \mu_1)C$ ,  $L \rightarrow L + \eta_1 S$

### 3.2 Codec 2: Vector Emission (Photon) — $M_2(f)$

$$M_2(f) = \begin{bmatrix} 1 & 0 & 0 & -a_2 & +b_2 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c_2 & 0 & 0 \\ 0 & 0 & 0 & 0 & d_2 & 0 \\ 0 & 0 & 0 & \varepsilon_2 f & 0 & 1 \end{bmatrix} \quad (3)$$

EM photon emission:

```
HANDLE_VECTOR:
; Immediate emission - no storage
IN AL, REG_ENTROPY      ; Energy ~ entropy
OUT CODEC2_VECTOR, AL    ; Emit photon
JMP EVENT_DONE
```

Matrix operation:  $S \rightarrow c_2 S$ ,  $C \rightarrow d_2 C$ ,  $L \rightarrow L + \varepsilon_2 f$

### 3.3 Codec 3: Tensor Delay (Graviton) — $M_3(g, \kappa)$

$$M_3(g, \kappa) = \begin{bmatrix} 1 & 0 & 0 & -a_3 & +b_3 & 0 \\ 0 & (1 + \alpha_3) & 0 & 0 & \beta_3 & 0 \\ 0 & 0 & (1 + \gamma_3) & 0 & 0 & 0 \\ 0 & 0 & 0 & (1 - \kappa g) & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 + \delta_3) & 0 \\ 0 & 0 & 0 & \varepsilon_3 g & 0 & 1 \end{bmatrix} \quad (4)$$

Gravitational tensor delay:

```
HANDLE_TENSOR:
; Delay proportional to curvature
IN AL, LOAD_ACCUMULATOR ; Gravitational load
MOV CL, AL               ; Use as delay counter
TENSOR_DELAY:
NOP                     ; Wait one cycle
LOOP TENSOR_DELAY       ; Delay ~ load

; Emit weak graviton
MOV AL, #0x01           ; Very low energy
OUT CODEC3_TENSOR, AL
JMP EVENT_DONE
```

**Key feature:** Tensor processing has cost 47 (vs scalar=1, vector=10), making gravity extremely weak. The NOP loop implements the quantization delay  $\Delta t_q \propto L$ .

### 3.4 Codec 4: Decoherence — $M_9(\Delta S)$

**Note:** Assembly CODEC4\_DECOHERE maps to matrix  $M_9$  (entropy increase).

$$M_9(\Delta S) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & (1 + \Delta S) & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 - \kappa_9 \Delta S) & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

Matrix operation:  $S \rightarrow (1 + \Delta S)S$ ,  $C \rightarrow (1 - \kappa_9 \Delta S)C$

### 3.5 Codec 5: Strong Confinement — $M_5(\Lambda_s, \chi)$

$$M_5(\Lambda_s, \chi) = \begin{bmatrix} 1 & 0 & 0 & -\alpha_5 & +\beta_5 & 0 \\ 0 & (1 + \rho_5) & 0 & 0 & \sigma_5 & 0 \\ 0 & 0 & (1 + \tau_5) & 0 & v_5 & 0 \\ 0 & 0 & 0 & (1 - \Lambda_s) & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 + \chi) & 0 \\ 0 & 0 & 0 & \varepsilon_5 \Lambda_s & 0 & 1 \end{bmatrix} \quad (6)$$

**Strong force confinement:**

```
HANDLE_CONFINEMENT:
; Energy trapped in internal oscillation
IN AL, CODEC5_CONFINEMENT
INC AL                      ; Increase internal energy
CMP AL, #0xF0              ; Confinement limit
JC CONFINEMENT_STORE

; Overflow - emit gluon burst
MOV AL, #0x80              ; Reset to half
CONFINEMENT_STORE:
OUT CODEC5_CONFINEMENT, AL ; Energy stays confined
JMP EVENT_DONE
```

**Key feature:** Energy circulates internally; emissions rare. Activation requires  $S > S_{\text{strong}}$  and  $r < r_{\text{confine}}$ .

### 3.6 Codec 6: Identity Rewrite (Weak) — $M_4(\theta_w, \Gamma_w)$

**Note:** Assembly CODEC6\_REWRITE maps to matrix  $M_4$  (weak interaction).

$$M_4(\theta_w, \Gamma_w) = \begin{bmatrix} \cos \theta_w & \sin \theta_w & 0 & -\alpha_4 & 0 & 0 \\ -\sin \theta_w & \cos \theta_w & 0 & 0 & \beta_4 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & (1 - \Gamma_w) & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 - \gamma_4) & 0 \\ 0 & 0 & 0 & \varepsilon_4 \Gamma_w & 0 & 1 \end{bmatrix} \quad (7)$$

**Weak force identity rewrite:**

```
HANDLE_REWRITE:
; Check identity budget
IN AL, REG_IDENTITY
CMP AL, #0x10              ; Minimum threshold
JNC EVENT_DONE            ; Sufficient

; Identity too low - rewrite
MOV AL, REG_IDENTITY
SHR AL, 1                  ; Id -> Id/2
OUT REG_IDENTITY, AL

; Emit neutrino byproduct
MOV AL, #0x02              ; Very low energy
OUT CODEC6_REWRITE, AL
JMP EVENT_DONE
```

**Matrix operation:** Identity–Phase rotation with parity violation. When identity budget insufficient, particle flavor changes.

## 4 Composite Codecs (7–10)

### 4.1 Codec 7: Collapse — $M_6(S) + M_9$

**Composite:** Instantaneous decoherence = rapid entropy increase + Pauli rewrite.

$$M_6(S) = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \rho_6 & 0 \\ 0 & 0 & 0 & S & 0 & 1 \end{bmatrix}, \quad \text{if } S > S_{\text{thr}} \quad (8)$$

Handler not implemented (placeholder NOP).

### 4.2 Codec 8: Black Hole — $M_3$ saturation + time dilation

When  $L \rightarrow L_{\text{max}}$ , tensor codec saturates capacity, triggering extreme time dilation:

**Black hole gate saturation:**

```
HANDLE_BLACKHOLE:
; Check gate overload
IN AL, LOAD_ACCUMULATOR
CMP AL, #0xF0          ; Near max (255)
JC EVENT_DONE

; Choke - slow time massively
CALL EXTREME_TIME_DILATION

; Hawking radiation leak
MOV AL, #0x01
OUT CODEC8_BLACKHOLE, AL
```

**Time dilation subroutine:**

```
EXTREME_TIME_DILATION:
IN AL, TIME_LAYER_BUS
MOV AL, #0x01          ; Nearly stop time
OUT TIME_LAYER_BUS, AL
RET
```

**Composite:** Not a fundamental codec but emergent from  $M_3$  capacity saturation.

### 4.3 Codec 9: Entanglement — $M_7$ phase correlation

**Composite:** Phase synchronization across spatially separated registers.

$$M_7(\omega, \phi_0) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & \cos(\omega\Delta t + \phi_0) & -\sin(\omega\Delta t + \phi_0) & 0 & \alpha_7 & 0 \\ 0 & \sin(\omega\Delta t + \phi_0) & \cos(\omega\Delta t + \phi_0) & 0 & \beta_7 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 + \gamma_7) & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (9)$$

**Entanglement phase sync:**

```
HANDLE_ENTANGLE:
; Check phase synchronization
IN AL, REG_PHASE
IN BL, REG_SPATIAL
XOR AL, BL          ; Compare phases
JZ MAINTAIN_ENTANGLE

; Phases don't match - break entanglement
MOV AL, #0x00
OUT CODEC9_ENTANGLE, AL
JMP EVENT_DONE

MAINTAIN_ENTANGLE:
MOV AL, #0xFF          ; Maintain sync
OUT CODEC9_ENTANGLE, AL
```

## 4.4 Codec 10: Cosmic Expansion

Handler not implemented. Would involve systematic scaling of spatial register across all particles.

# 5 Special Mechanisms

## 5.1 Pauli Exclusion (Codec 6 Trigger)

Pauli collision detection:

```
PAULI_CHECK:
    ; Hash quantum state
    IN AL, REG_IDENTITY
    IN BL, REG_PHASE
    IN CL, REG_SPATIAL
    XOR AL, BL
    XOR AL, CL                ; State hash

    ; Check collision
    CMP AL, #0x00
    JZ PAULI_VIOLATION
    RET

PAULI_VIOLATION:
    ; Force delay + phase increment
    CALL WAIT_ONE_TICK
    IN AL, REG_PHASE
    INC AL
    OUT REG_PHASE, AL
    RET
```

This implements  $M_6$  trigger condition: when two fermions have identical  $(\Phi, X, C)$  states (hash collision), one must apply identity flip.

## 5.2 Fermion $720^\circ$ Phase Cycle

Spinor double-cover behavior:

```
FERMION_SPIN_UPDATE:
    IN AL, REG_PHASE
    INC AL
    CMP AL, #0x68                ; 360 degrees
    JNE CHECK_720

    ; At 360: flip spatial, keep phase
    IN BL, REG_SPATIAL
    XOR BL, #0xFF                ; Spatial flip
    OUT REG_SPATIAL, BL
    JMP STORE_PHASE

CHECK_720:
    CMP AL, #0xD0                ; 720 degrees
    JNE STORE_PHASE

    ; At 720: full reset
    MOV AL, #0x00
    MOV BL, #0x00
    OUT REG_SPATIAL, BL

STORE_PHASE:
    OUT REG_PHASE, AL
    RET
```

This encodes the  $SU(2)$  double cover: fermions require 720 to return to initial state.

| Matrix | Assembly Codec      | Address    | Status                    |
|--------|---------------------|------------|---------------------------|
| $M_1$  | CODEC1_SCALAR       | 0x0010     | Implemented               |
| $M_2$  | CODEC2_VECTOR       | 0x0011     | Implemented               |
| $M_3$  | CODEC3_TENSOR       | 0x0012     | Implemented               |
| $M_4$  | CODEC6_REWRITE      | 0x0015     | Implemented               |
| $M_5$  | CODEC5_CONFINE      | 0x0014     | Implemented               |
| $M_6$  | PAULI_CHECK         | subroutine | Implemented               |
| $M_7$  | CODEC9_ENTANGLE     | 0x0018     | Composite                 |
| $M_8$  | (spatial advection) | implicit   | In main loop              |
| $M_9$  | CODEC4_DECOHERE     | 0x0013     | Implemented               |
| —      | CODEC7_COLLAPSE     | 0x0016     | Composite ( $M_6 + M_9$ ) |
| —      | CODEC8_BLACKHOLE    | 0x0017     | Composite ( $M_3$ sat.)   |
| —      | CODEC10_COSMIC      | 0x0019     | Placeholder               |

Table 4: Complete codec mapping verification

## 6 Consistency Cross-Check

### 6.1 Matrix-to-Assembly Mapping

### 6.2 Architectural Constants

| Constant            | Assembly Value | Physical Meaning               |
|---------------------|----------------|--------------------------------|
| PLANCK_TICK         | 0x0001         | $t_P = 5.39 \times 10^{-44}$ s |
| CRIT_ENTROPY        | 0x006C (108)   | Cosmic reset threshold         |
| PHASE_720           | 0x02D0 (720)   | Fermion full cycle             |
| Higgs mass          | 0x7D (125)     | 125 GeV (internal units)       |
| Coherence threshold | 0x80 (128)     | 50% of max                     |
| Confinement limit   | 0xF0 (240)     | 94% of capacity                |

Table 5: Assembly constants and physical correspondence

### 6.3 Execution Sequence Consistency

Matrix algebra execution rule:

$$\mathbf{R}_{\text{new}} = \prod_{i \in \mathcal{A}} M_i(\text{params}_i) \mathbf{R}_{\text{old}} \quad (10)$$

Assembly polling loop:

1. Poll all 10 codec addresses (0x0010–0x0019)
2. Test event flag (bit 7)
3. Jump to handler for active codecs
4. Each handler reads/modifies working registers (0x0020–0x0024, 0x0002)
5. Return to main loop

The assembly implements sequential matrix application through register reads/writes. Non-commutativity  $[M_i, M_j] \neq 0$  encoded in handler execution order.



## 6.4 Critical Distinction: Registers vs Codecs

The six-register computational substrate has two distinct layers:

### Layer 1: Hardware Registers (The State)

$$\mathbf{R} = [\text{Id}, \Phi, X, S, C, L]^T \in \mathbb{R}^6$$

These are the **actual storage locations**: `REG_IDENTITY`, `REG_PHASE`, `REG_SPATIAL`, `REG_ENTROPY`, `REG_COHERENCE`, `LOAD_ACCUMULATOR`.

### Layer 2: Codec Operators (The Transformations)

$$\mathbf{R}_{\text{new}} = M_{\text{codec}} \mathbf{R}_{\text{old}}$$

These are **operations** that transform the registers: `CODEC1_SCALAR`  $\rightarrow M_1$ , `CODEC2_VECTOR`  $\rightarrow M_2$ , etc.

**Key point:** Codecs are *not registers*—they are subroutines that manipulate the six registers. Addresses `0x0010–0x0019` are event flags/operation queues, not storage.

**Analogy:** Registers are variables; codecs are functions that operate on those variables.

## 7 Performance Characteristics

### 7.1 Cost Hierarchy

From matrix algebra: scalar:vector:tensor = 1:10:47

Assembly evidence:

- `HANDLE_SCALAR`: 7 instructions
- `HANDLE_VECTOR`: 3 instructions (immediate emission)
- `HANDLE_TENSOR`: 4 + NOP loop (delay  $\propto$  load)

The tensor delay loop encodes the  $47\times$  computational cost: longer processing time per graviton.

### 7.2 Throughput Limit

Capacity:  $10^6$  packets per Planck tick (from matrix derivation)

Assembly: Load accumulator is 8-bit (`AL`), max value 255 = `0xFF`

Mismatch: Assembly simplified for demonstration. Full implementation would use 20-bit accumulator to represent  $10^6$  capacity.

## 8 Conclusion

The assembly language `UNIVERSE.ASM` provides executable semantics for the six-register computational substrate:

1. **Six fundamental codecs** ( $M_1$  through  $M_6$ , plus  $M_7$ – $M_9$ ) map to assembly handlers
2. **Codecs 7–10** are architectural composites, not primitives
3. **Register polling loop** implements matrix product  $\prod M_i$
4. **Pauli exclusion** via hash collision detection
5. **Fermion spinors** via  $720^\circ$  phase cycle
6. **Gravity weakness** via tensor delay (cost 47)

The assembly reveals that **physics is a polling loop** at Planck frequency with deterministic, synchronous codec evaluation. No interrupts, no randomness—just registers, gates, and a clock.

**Welcome to the machine.**